UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

# Criptografía y Seguridad

## Tarea 1: Cifrados

*Profesor:* José Antonio Macías García

*Ayudante:* Juan Angeles Hernández

*Ayudante:* Erick Bernal Márquez

**Alumno:** Gustavo Alonso Domínguez Sánchez

418047121

10 de Marzo de 2026

# 1. Problemas

## 1.1. Cifrado César (a)

Para la resolución de este problema se utiliza el algoritmo de descifrado (César) usando la aritmética modular. Dado que la instrucción exige incluir la 'ñ', nuestro alfabeto pasa a ser uno de 27 caracteres. Por lo tanto, cualquier desplazamiento que hagamos estará delimitado por un módulo de 27. El índice de cada letra irá del 0 (para la 'a') al 26 (para la 'z').
La implementación es de la siguiente forma:

- Función principal `descifrar_cesar`: Se creó una única función que contiene la lógica de rotación del texto. Primero, convierte todo a minúsculas para estandarizar. Después aplica la fórmula de descifrado a todas las letras del texto: busca el índice de la letra cifrada en el alfabeto, y le resta la clave, para luego aplicar módulo 27, garantizando la rotación circular, obteniendo así el índice de la letra rotada y la letra en sí buscándola en el alfabeto. Repetimos para toda el texto, obteniendo así el texto rotado.

- Fuerza Bruta: Un simple ciclo `for` que itera 27 veces llamando a la función principal. Por medio de la visualización se descubre que el texto correcto es de la clave 2.

- Conocimiento Adicional: Al restar el índice de la 'o' al de la 'd' y aplicar módulo, obtenemos la clave exacta sin iterar.

- Se utilizó un diccionario para almacenar cada letra como "llave" y sus apariciones en el texto cifrado como "valor". Luego, un ciclo `for` encuentra la letra de mayor aparición. Al encontrar que la 'j' es la que más se repite, asumimos que equivale a la 'e' y aplicamos la misma resta modular del paso anterior para encontrar la clave.

El código de este programa se encuentra en el archivo `p1_cifrado_cesar_a.py`.

## 1.2. Cuadrado de Polibio

El Cuadrado de Polibio es un cifrado de sustitución clásico que sustituye cada letra en dos componentes numéricos. Se basa en acomodar un alfabeto en una matriz bidimensional (cuadrícula). Para cifrar una letra, se busca su posición en la cuadrícula y se devuelven sus coordenadas (generalmente Fila y luego Columna).

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | a | b | c | d | e |
| 2 | f | g | h | i | j |
| 3 | k | l | m | n | o |
| 4 | p | q | r | s/z | t |
| 5 | u | v | w | x | y |

Figura 1: Cuadrado de Polibio utilizado para el programa.

En particular, el programa implementa el algoritmo con:

- Matriz constante `CUADRADO_POLIBIO`: Se define una lista que contenga 5 sub-listas, y en donde cada sub-lista (fila) tiene 5 caracteres (columnas), representando el tablero criptográfico.
- Función `limpiar_texto`: Es la función de pre-procesamiento y estandarización. Se encarga de pasar todo a minúsculas, aplicar los reemplazos (resolvemos lo de los acentos y convertimos la z en s), y elimina signos y espacios, dejando una cadena de texto lista para encriptar.
- Función `encriptar_polibio`: Usa ciclos anidados (uno para las filas y otro para las columnas) para buscar en la matriz completa la letra actual. Una vez que la encuentra, concatena las coordenadas sumando +1 (ya que los índices en Python comienzan desde 0).
- Función `desencriptar_polibio`: Hace lo inverso de lo anterior. Toma cada par de dígitos, los convierte a enteros, les resta 1 ()para volver a índices de Python), y extrae directamente la letra de la matriz.

El código de este programa se encuentra en el archivo textttp2_cuadrado_polibio.py.

### 1.3. Cifrado Vigenère

El cifrado Vigenère es polialfabético, lo que significa que una misma letra puede cifrarse de múltiples formas dependiendo de la clave. Esto hace inútil el análisis de frecuencias que usamos en el cifrado César. Sin embargo, Kasiski notó una debilidad fundamental: si una misma palabra o fragmento en el texto original se cifra usando la misma parte de la clave, el texto cifrado resultante será idéntico. El Test de Kasiski consiste en buscar secuencias de letras repetidas en el texto cifrado (generalmente de 3 o más letras, llamadas trigramas o n-gramas); contar la distancia (cantidad de letras) que hay entre estas repeticiones; como estas repeticiones ocurren porque la clave se repite y coincidió con la misma palabra, la distancia entre las repeticiones debe ser un múltiplo de la longitud de la clave. Al obtener los divisores comunes (o el Máximo Común Divisor) de todas las distancias encontradas, podemos deducir la longitud probable de la clave. Para resolver este problema, se realizó en el programa lo siguiente:

- Limpieza de datos: Quitar signos y espacios. Se elimina cualquier carácter que no sea una letra de la 'A' a la 'Z'.
- Búsqueda de patrones (trigramas): Crearemos un ciclo que recorra el texto tomando bloques de 3 letras (trigramas). Usaremos un diccionario para guardar en qué posiciones (índices) aparece cada trigrama. Solo consideraremos los trigramas que aparece más de una vez, ya que estos son los de utilidad.
- Cálculo de distancias: Para los patrones que se repiten, calcularemos la distancia matemática restando la posición de la primera aparición de la posición de la segunda (y así sucesivamente).
- Divisores: Para cada distancia calculada, encontraremos todos sus factores o divisores mayores a 2 (por ejemplo, si la distancia es 20, sus divisores útiles son 4, 5, 10, 20).
- Análisis de frecuencias de los divisores: Contaremos qué divisor es el que más se repite en todas las distancias de todos los patrones. El divisor (mayor a 2) que más se repita nos dará la longitud más probable de la clave.

El código de este programa se encuentra en el archivo `p3_vigenere_a.py`.

### 1.4. Cifrado César (b)

Dado que ambos criptogramas son demasiado cortos para realizar un análisis de frecuencias, el método más directo y efectivo es el ataque por fuerza bruta. Considerando un alfabeto de 27 letras ('ñ' incluida), evaluaremos todos los desplazamientos hasta encontrar la palabra con sentido lógico en español. Recordemos que la fórmula de desencriptación es:

$$D_i = (C_i - k) \bmod 27 \tag{1}$$

donde:

$D_i$: es la i-ésima letra desencriptada o sustituida después de la rotación

$C_i$: es la i-ésima letra encriptada

$k$: es el desplazamiento a aplicar

- Primer mensaje: ***dqiqo***

  Realizamos una tabla de desplazamientos sucesivos a la cadena encriptada:

| $C_i \backslash K$ | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u | v | w | x | y | z | a | b | c |
| q | r | s | t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p |
| i | j | k | l | m | n | ñ | o | p | q | r | s | t | u | v | w | x | y | z | a | b | c | d | e | f | g | h |
| q | r | s | t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p |
| o | p | q | r | s | t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ |

Figura 2: Fuerza bruta de descifrado César para la primer cadena.

  La opción con más sentido es el texto con la palabra ***humus*** (suponiendo que el mensaje tenga un significado en español). La clave es **23**.

- Primer mensaje: ***btvtvshz***

  Aplicamos la misma técnica de fuerza bruta. Al ser un texto más largo, el descarte visual de palabras sin sentido es más rápido.

| $C_i \backslash K$ | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u | v | w | x | y | z | a |
| t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | z |
| v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u |
| t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s |
| v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u |
| s | t | u | v | w | x | y | z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r |
| h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u | v | w | x | y | z | a | b | c | d | e | f | g |
| z | a | b | c | d | e | f | g | h | i | j | k | l | m | n | ñ | o | p | q | r | s | t | u | v | w | x | y |

Figura 3: Fuerza bruta de descifrado César para la segunda cadena.

Se revela una frase coherente con la clave es **7**. La frese original es ***unonomas***.